Automatants



H-Net : apprendre à découper les séquences



Au programme

1. C’est quoi un H-Net ?
2. Pourquoi ?
3. Comment ça fonctionne ?
4. Résultats & mise à l’échelle

I. C’est quoi un H-Net ?

Dynamic Chunking for End-to-End Hierarchical Sequence Modeling

Sukjun Hwang, Brandon Wang et al. (2025)

Un modèle hiérarchique qui apprend à regrouper ses entrées pendant son entraînement.

Pour le texte : des octets en entrée et en sortie, sans tokenizer BPE externe.

Source : arXiv:2507.07955

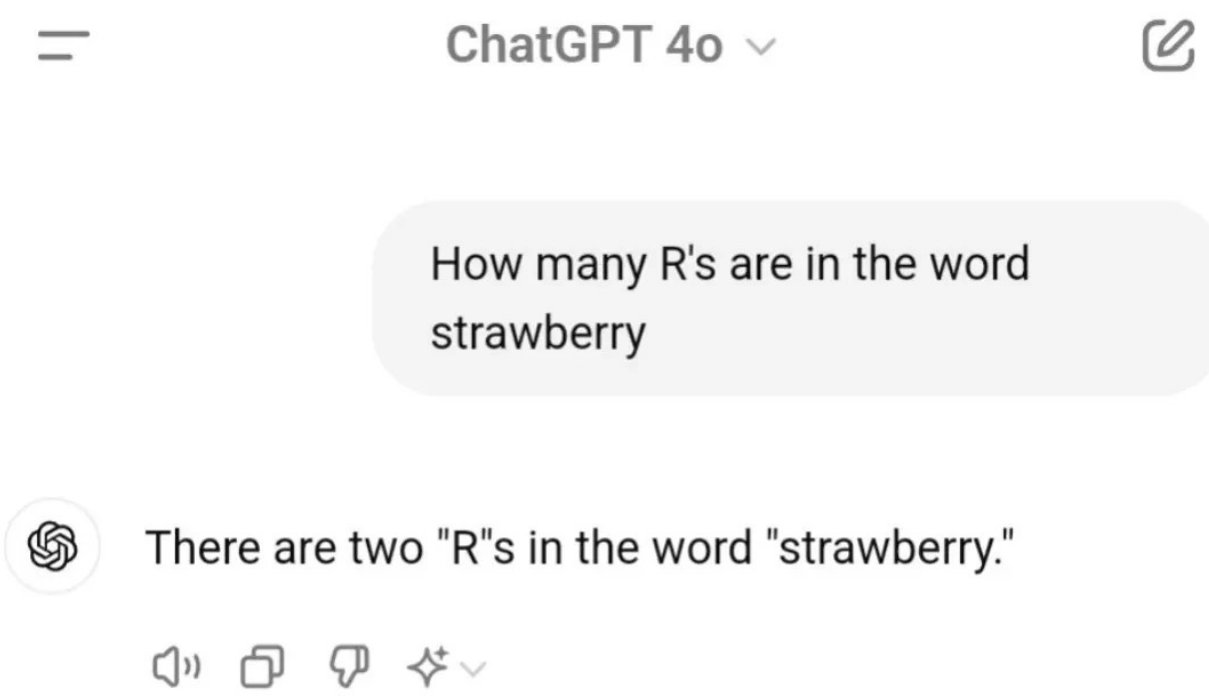
II. Pourquoi ?

Combien de « r » dans strawberry ?

S T R A W B E R R Y

Réponse : 3

Le découpage en tokens peut rendre les tâches sur les caractères moins naturelles. Ce n’est pas la seule cause possible des erreurs.



BPE : comment ça marche ?

1. Partir d’unités élémentaires, souvent des octets.
2. Sur un corpus, fusionner les paires adjacentes les plus fréquentes, puis répéter.
3. Appliquer les fusions apprises pour encoder de nouveaux textes.

Exemple simplifié : fusions apprises sur un corpus

b a n a n a → b an an a → b anan a

a + n = an

an + an = anan

Le vocabulaire est fixe après apprentissage. La compression varie selon la langue et le domaine ; les tâches sur les caractères peuvent rester difficiles.

Avant H-Net : plusieurs pistes

Octets directement : séquences longues et calcul coûteux.

Hourglass et MEGABYTE : compression à granularité fixe.

SpaceByte : frontières guidées par des heuristiques.

Dynamic token pooling : apprentissage des décisions difficile à stabiliser.

Hwang et al., sections 1 et A

The Bitter Lesson

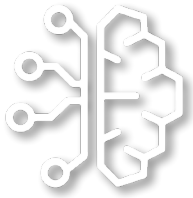
« General methods that leverage computation are ultimately the most effective, and by a large margin. »

Richard Sutton, The Bitter Lesson (2019)

H-Net applique cette idée à la segmentation : apprendre la découpe avec le modèle.



The Bitter Lesson (2019) | Photo : Numenta, Wikimedia Commons (recadrée), source · CC BY 3.0



III. Comment ça fonctionne ?

Entrée : une séquence d’octets.

Le modèle apprend où placer les frontières selon le contexte.

Sortie : prédiction du prochain octet.

La segmentation et la prédiction sont optimisées ensemble.

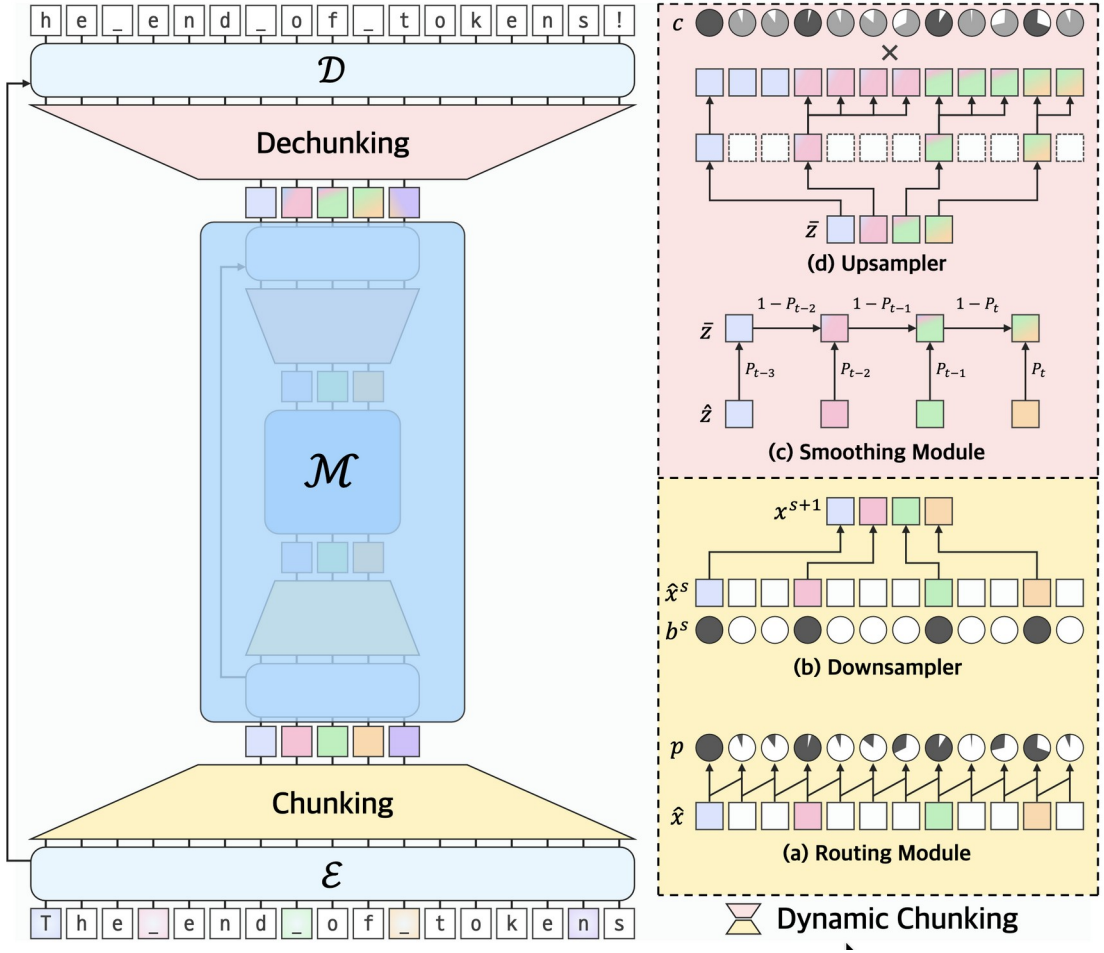
Une hiérarchie inspirée de U-Net

L’encodeur traite les détails.

Le chunking réduit la longueur de séquence.

Le réseau principal traite les représentations compressées.

Le décodeur retrouve la résolution initiale.



Hwang et al., figure 1

Architecture : encodeur, réseau principal, décodeur

E, M et D sont des modèles de séquence : Transformer ou SSM.

$$\hat{x}^s = \mathcal{E}^s(x^s), \quad \hat{z}^S = \mathcal{M}(x^S), \quad \hat{z}^s = \mathcal{D}^s(z^s) \tag{1}$$

$$(x^{s+1}, p^s) = \mathrm{Chunk}(\hat{x}^s) \tag{2}$$

$$z^s = \mathrm{Dechunk}(\hat{z}^{s+1}, p^s) + \mathrm{Linear}(\hat{x}^s) \tag{3}$$

La voie résiduelle préserve les détails fins.

Hwang et al., équations 1–3

Chunking dynamique : choisir les frontières

Le routeur compare deux représentations voisines.

$$q_t = W_q \hat{x}_t, \qquad k_t = W_k \hat{x}_t$$

$$p_t = \frac{1}{2}\left(1 - \frac{q_t^\top k_{t-1}}{\|q_t\| \, \|k_{t-1}\|}\right), \qquad b_t = \mathbf{1}_{\{p_t \geq 0.5\}} \qquad (4)$$

Le downsampler conserve les positions où $b_t = 1$. La première position est toujours une frontière.

Hwang et al., équation 4

Dechunking : lisser puis étendre

Le smoothing mélange le chunk courant et la représentation précédente.

$$\bar{z}_t = P_t \hat{z}_t + (1 - P_t)\bar{z}_{t-1} \qquad (5)$$

P_t élevé : priorité au nouveau chunk. P_t incertain : une partie du contexte précédent est conservée.

Cette interpolation fournit un chemin de gradient vers le routeur.

Hwang et al., équation 5

Upsampling : retrouver chaque position

$$c_t = p_t^{b_t}(1 - p_t)^{1-b_t} \tag{6}$$

$$\mathrm{STE}(c_t) = c_t + \mathrm{stopgradient}(1 - c_t) \tag{7}$$

$$\widetilde{z}_t = \bar{z}_{\sum_{k=1}^{t} b_k} \tag{8}$$

$$\mathrm{Upsampler}(\bar{z}, c)_t = \mathrm{STE}(c_t) \cdot \widetilde{z}_t \tag{9}$$

La somme compte les frontières déjà rencontrées : elle donne l'indice du chunk courant. STE vaut 1 au passage avant et transmet un gradient via c_t.

Hwang et al., équations 6–9

Ratio loss : contrôler la compression

F : fraction de positions retenues. G : score moyen de frontière.

$$\mathcal{L}_{\text{ratio}} = \frac{N}{N-1}\Big((N-1)FG + (1-F)(1-G)\Big) \qquad (10)$$

$$F = \frac{1}{L}\sum_{t=1}^{L} b_t, \qquad G = \frac{1}{L}\sum_{t=1}^{L} p_t$$

La cible est une frontière tous les N éléments en moyenne. La loss auxiliaire ajuste G selon F, tout en laissant la découpe adaptative.

Hwang et al., équation 10 ; α = 0,03 dans leurs expériences

Trois techniques pour stabiliser le modèle

RMSNorm après E, M et D : équilibre la voie profonde et la voie résiduelle.

Deux voies : projection apprise uniquement sur la connexion résiduelle ; chemin vers le réseau profond préservé.

Learning rate : adapté à la taille et au nombre de positions traitées à chaque niveau.

Hwang et al., section 2.3

IV. Résultats : moins de calcul

À loss égale en validation, H-Net utilise 22 % de calcul en moins.

Les courbes comparent un H-Net à un Transformer BPE sur les mêmes données, en bits par octet.

Résultat du travail ultérieur de Cartesia, avec une architecture améliorée.

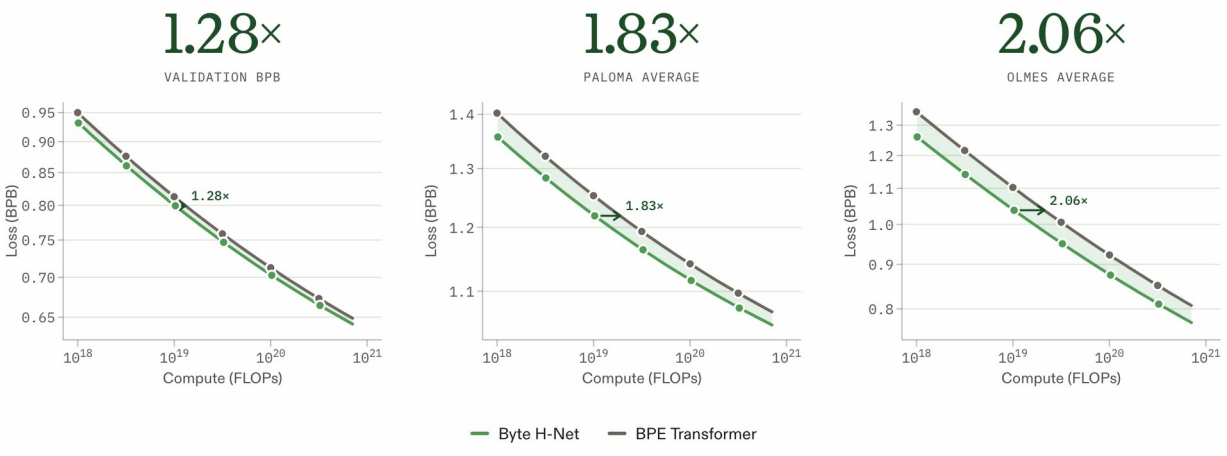


Figure 4. Compute-optimal scaling of the BPE Transformer ladder and matched H-Net models. The H-Net reaches the BPE Transformer's loss with 22% less compute on held-out pretraining data (validation bpb), and the gap widens on downstream evaluation suites.

Cartesia, 2026, figure 4

Sur du code Python : 2,24×

À loss égale sur Python, le Transformer BPE demande environ 2,24 fois plus de calcul.



Cartesia, 2026, figure 5

Mise à l'échelle : comparaison avec SmolLM3

Le H-Net atteint un score moyen comparable sur six tâches après environ 1 000 milliards de tokens équivalents, contre 11 000 milliards pour SmolLM3.

Les recettes et paramètres diffèrent : les auteurs discutent les limites du contrôle des données.

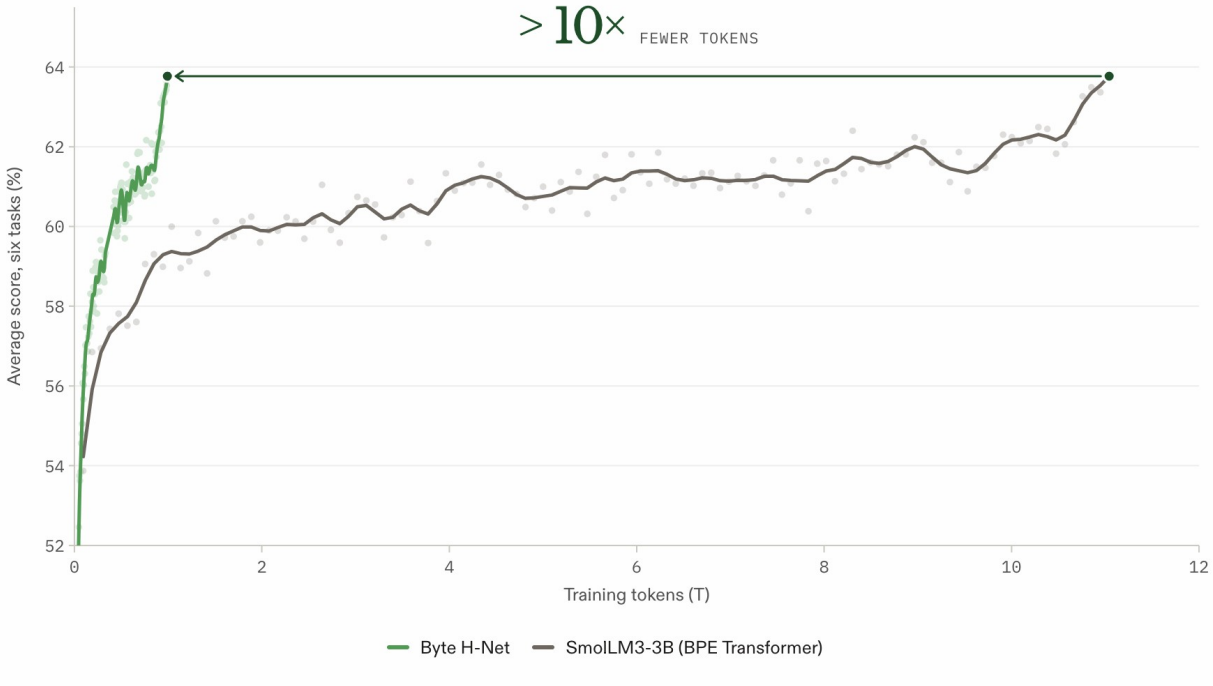


Figure 7. Byte-level 2-stage H-Net vs. SmolLM3 (compute and data matched) on pretraining evaluations.

Cartesia, 2026, figure 7

Au-delà du texte : ADN et images

La hiérarchie peut traiter d’autres séquences que du texte.

ADN : résultats de modélisation dans l’article original.

Images : résultats exploratoires présentés par Cartesia.

Une même famille d’architecture peut servir plusieurs modalités ; cela ne démontre pas à lui seul un modèle multimodal conjoint.

Hwang et al., section 3.2 ; Cartesia, partie 4

Sources

Hwang et al. — Dynamic Chunking for End-to-End Hierarchical Sequence Modeling

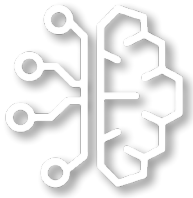
https://arxiv.org/abs/2507.07955

Cartesia — From Tokens to Bytes, Part 1

https://www.cartesia.ai/blog/from-tokens-to-bytes-part-1

Vidéo : https://www.youtube.com/watch?v=XmsgfvBqLXA

Richard Sutton — The Bitter Lesson (2019)



Questions ?

Merci pour votre attention !